

TeamForge

SDE 1 Interview Preparation Guide

How to use this guide: Each answer is structured using either the **STAR method** (Situation, Task, Action, Result) for behavioral/engineering stories or the **PREP method** (Point, Reason, Evidence, Point) for technical design choices. Follow-up questions provide substantive 2-3 sentence answers matching interviewer expectations. All answers are grounded in the actual TeamForge codebase.

Project-Specific Technical Questions

● **HIGH (~95%) Q1: Tell me about your project - what problem does TeamForge solve?**

■ **Project Architecture & Scope | STAR Framework**

S (Situation): Small-to-medium teams often struggle with bloated project management tools like Jira or unstructured channels like chat and spreadsheets, leading to lost tasks and unclear ownership.

T (Task): I designed and built TeamForge, a multi-tenant B2B project management SaaS that delivers structured organization without operational bloat.

A (Action): I implemented a 3-tier hierarchy (Workspaces -> Projects -> Tasks) with strict data isolation per workspace. Built the full backend using Node.js, Express, TypeScript, and MongoDB, paired with a React 18 + Vite frontend using TanStack Query for data fetching and Zustand for state persistence.

R (Result): Delivered a production-ready application supporting secure multi-tenancy, instant team onboarding via invite codes, and workspace-scoped Role-Based Access Control (RBAC).

Follow-up Questions & Interviewer-Expected Answers:

Q: Why choose multi-tenancy at the workspace level rather than database isolation?

A: Workspace-level isolation using a shared database with workspaceId tenant scoping offers the best trade-off between cost efficiency and complexity for an MVP. Database-per-tenant adds significant infrastructure overhead and connection pool exhaustion at small scale, whereas indexing workspaceId across collections ensures fast queries and logical isolation.

Q: How do invite codes simplify onboarding without compromising security?

A: Workspace owners generate secure UUID invite codes. When a user registers or logs in with an invite code, our backend transaction bypasses public workspace creation and atomically joins the user directly to the target workspace with a default MEMBER role.

Key Takeaway: 3-tier multi-tenant architecture (Workspace -> Project -> Task) with tenant-scoped MongoDB queries.

● **HIGH (~90%) Q2: How does authentication work in TeamForge? Walk me through the request lifecycle.**

■ **Authentication & Security | PREP Framework**

P (Point): TeamForge implements Passport.js with three authentication strategies: JWT (cookie-based), Google OAuth 2.0, and Local email/password.

R (Reason): This multi-strategy approach accommodates different login workflows while ensuring session security. Storing JWTs in httpOnly cookies eliminates XSS token theft compared to localStorage.

E (Evidence):

On login, `signJwtToken` creates an HS256-signed token (audience: ['user'], 24h expiry) attached to an `httpOnly`, `SameSite=Strict` cookie. On requests, `passportAuthenticationJWT` extracts the token, verifies the user in MongoDB, and populates `req.user`. For Google OAuth, `loginOrCreateAccountService` runs inside a MongoDB transaction to link OAuth provider IDs to User documents.

P (Point): This provides a seamless, secure authentication pipeline across both social and credentials-based logins.

Code Anchor:

```
// backend/src/middlewares/isAuthenticated.middleware.ts
const isAuthenticated = async (req: Request, res: Response, next: NextFunction) => {
  if (!req.user || !req.user._id) throw new UnauthorizedException('Unauthorized. Please log in.');
```

Follow-up Questions & Interviewer-Expected Answers:

Q: Why store JWT in httpOnly cookies instead of localStorage?

A: Tokens stored in `localStorage` are vulnerable to Cross-Site Scripting (XSS) because any injected client script can read `localStorage`. `HttpOnly` cookies are inaccessible to client JS, meaning XSS attacks cannot extract session tokens.

Q: How does account linking work if a user registers with email/password and later uses Google OAuth?

A: When a Google OAuth login occurs, `loginOrCreateAccountService` checks if a User exists with that email. If found, it creates an Account document linking provider: 'GOOGLE' and providerId to the existing User ID inside a transaction, enabling unified login.

Key Takeaway: Passport.js + JWT in `httpOnly` cookies + MongoDB transactional account linking.

● HIGH (~85%) Q3: Explain your Role-Based Access Control (RBAC) system. How are permissions enforced?

■ Authorization & RBAC | STAR Framework

S (Situation): In a multi-tenant platform, a user's permissions must vary by context - a user can be an OWNER in Workspace A but only a MEMBER in Workspace B.

T (Task): I needed a granular authorization system that evaluates permissions per request without coupling roles globally to the User model.

A (Action): I created a 3-part RBAC model: Role collection (OWNER, ADMIN, MEMBER), Member join table (associating `userId`, `workspaceId`, `role`), and RolePermissions mapping 14 granular permissions (e.g., `CREATE_TASK`, `DELETE_WORKSPACE`). I wrote a reusable `roleGuard` middleware to enforce permissions at the route layer.

R (Result): Achieved zero-trust request authorization. Permissions are dynamically evaluated based on the user's active workspace membership.

Code Anchor:

```
// backend/src/utils/roleGuard.ts
export const roleGuard = (role: keyof typeof RolePermissions,
  requiredPermission: PermissionType[]) => {
  const permission = RolePermissions[role];
  const hasPermission = requiredPermission.every((p) => permission.includes(p));
  if (!hasPermission) throw new UnauthorizedException('Permission denied');
```

Follow-up Questions & Interviewer-Expected Answers:

Q: How does `roleGuard` enforce required permissions at the endpoint level?

A: `roleGuard` receives the user's member role and an array of required `PermissionType` enums. It looks up `RolePermissions[role]` and executes `.every(per => userPermissions.includes(per))`. If permission is lacking, it immediately throws `UnauthorizedException` before the controller runs.

Q: How does the frontend stay synchronized with backend RBAC rules?

A: The AuthProvider fetches the current workspace and user membership, utilizing the usePermissions hook to derive active permissions. The HOC wraps UI elements (like 'Delete Workspace' buttons) to conditionally render them based on hasPermission().

Key Takeaway: Workspace-scoped RBAC via Member join table and roleGuard middleware.

● **HIGH (~80%) Q4: Describe your database schema - what are the core collections and relationships?**

■ Database Schema & Modeling | PREP Framework

P (Point): TeamForge's database consists of 7 collections: Users, Accounts, Workspaces, Members, Roles, Projects, and Tasks.

R (Reason): This normalized collection structure cleanly separates user identities, workspace boundaries, and project management entities while maintaining fast retrieval paths.

E (Evidence):

- 1) Users holds identity & currentWorkspace reference.
- 2) Accounts links OAuth providers (userId, provider, providerId).
- 3) Workspaces represents tenants (name, owner, inviteCode).
- 4) Members is the join collection (userId, workspaceId, role).
- 5) Projects belongs to workspace and createdBy.
- 6) Tasks links project, workspace, createdBy, and assignedTo.
- 7) Roles defines permission arrays.

P (Point): By keeping relationships explicit and referencing IDs, we preserve referential integrity and simplify tenant isolation.

Follow-up Questions & Interviewer-Expected Answers:

Q: Why does the Task model store a direct workspace reference when it already belongs to a project?

A: This is intentional denormalization. Storing workspace directly on Task allows task filtering, pagination, and analytics queries to filter by workspace without performing costly \$lookup joins through the Project collection first.

Q: How do you prevent a user from assigning a task to someone outside the workspace?

A: In createTaskService, before saving, we query MemberModel.exists({ userId: assignedTo, workspaceId }). If the assigned user is not an active workspace member, the service throws NotFoundException('Assigned user is not a member of the workspace').

Key Takeaway: 7 collections connected via relational ObjectIds with denormalized tenant keys.

● **HIGH (~75%) Q5: When and why do you use MongoDB transactions in TeamForge?**

■ Data Consistency & Transactions | STAR Framework

S (Situation): Multi-document mutations - such as deleting a workspace or registering a user via invite code - risk leaving orphan records or partial state if the server crashes mid-execution.

T (Task): I needed guarantees that multi-step operations complete atomically (all-or-nothing).

A (Action): I utilized MongoDB Sessions (mongoose.startSession()) and ACID transactions across critical services. In deleteWorkspaceByIdService, the transaction deletes all associated Projects, Tasks, Members, resets user workspace state, and deletes the Workspace document within a single session.

R (Result): Guaranteed 100% data consistency. Failed transactions auto-rollback via session.abortTransaction(), preventing dangling data.

Follow-up Questions & Interviewer-Expected Answers:

Q: What happens if a database session times out or encounters a write conflict during a transaction?

A: MongoDB throws an error, triggering our catch block which calls `session.abortTransaction()` and passes the error to `errorHandler.middleware.ts`. The transaction guarantees that no partial changes persist.

Q: Why are transactions not used for every database write in TeamForge?

A: Transactions introduce lock overhead and latency because MongoDB must maintain write-intent logs across replica nodes. We reserve transactions strictly for multi-document operations where partial failure damages data integrity (e.g., workspace teardown, registration).

Key Takeaway: ACID compliance for multi-document mutations using Mongoose sessions.

● **MEDIUM (~70%) Q6: How is your backend structured, and how do you handle asynchronous errors?**

■ **Backend API Architecture | PREP Framework**

P (Point): TeamForge uses an Express + TypeScript MVC pattern (Routes -> Controllers -> Services) with Zod validation and a custom `asyncHandler` wrapper.

R (Reason): Separating HTTP handling from business logic keeps code modular and testable, while `asyncHandler` eliminates repetitive try-catch blocks across controllers.

E (Evidence):

`asyncHandler` wraps controller functions: `(req, res, next) => Promise.resolve(fn(req, res, next)).catch(next)`. Controllers parse request bodies via Zod schemas, then delegate to services (`task.service.ts`, `workspace.service.ts`). Thrown custom exceptions (`BadRequestException`, `NotFoundException`) inherit from `AppError` and are caught by `errorHandler.middleware.ts`.

P (Point): This creates a predictable, clean request lifecycle with centralized error handling.

Follow-up Questions & Interviewer-Expected Answers:

Q: How does `errorHandler.middleware.ts` format error responses for the client?

A: The error middleware inspects the error object. If it is an instance of `AppError`, it extracts `statusCode` (e.g., 400, 404, 401) and `errorCode` enum, returning `{ message, errorCode, statusCode }`. Uncaught runtime errors default to 500 `INTERNAL_SERVER_ERROR`.

Q: What role does Zod play in controller request handling?

A: Zod schemas enforce strict runtime validation on `req.body`, `req.params`, and `req.query`. If validation fails, Zod throws a formatted validation error before service logic runs, preventing invalid data from entering the database.

Key Takeaway: Async error propagation via `asyncHandler` and centralized `AppError` formatting.

● **MEDIUM (~65%) Q7: How do you handle client state vs. server state on the frontend?**

■ **Frontend State & Data Fetching | PREP Framework**

P (Point): We separate server state using TanStack React Query v5 from client state using Zustand and React Context.

R (Reason): Mixing server cache with UI state leads to sync bugs and stale data. React Query manages async caching, background revalidation, and loading states, while Zustand holds persistent client state.

E (Evidence):

Server data (workspaces, projects, tasks, member lists) is managed by custom React Query hooks (`useGetWorkspaceQuery`, `useGetAllTasksQuery`). On mutations (e.g., `createTaskMutationFn`), `queryClient.invalidateQueries` triggers targeted refetches. Client state (active workspace selection, modal visibility) is managed by Zustand with `sessionStorage` persistence.

P (Point): This separation eliminates manual state syncing and ensures optimal UI rendering speed.

Follow-up Questions & Interviewer-Expected Answers:

Q: How does React Query handle query invalidation when a task is created or edited?

A: Upon successful mutation in useMutation, the onSuccess callback calls queryClient.invalidateQueries({ queryKey: ['all-tasks', workspaceId] }). This marks cached task lists as stale and triggers a silent background refetch.

Q: Why use sessionStorage persistence for Zustand instead of localStorage?

A: Workspace selection and temporary session states are session-bound. sessionStorage automatically resets when a tab closes, preventing cross-session workspace leaks if multiple users use the same browser.

Key Takeaway: TanStack React Query for async server cache + Zustand for local UI state.

● MEDIUM (~60%) Q8: How do you enforce end-to-end type safety between backend and frontend?

■ Type Safety & Interfaces | STAR Framework

S (Situation): In full-stack JavaScript/TypeScript projects, API contract mismatches between backend JSON responses and frontend component props cause subtle runtime bugs.

T (Task): I established an end-to-end typing strategy across Mongoose models, API response payloads, and React components.

A (Action): On the backend, I authored strict Mongoose Document interfaces (UserDocument, TaskDocument). On the frontend, I centralized all contract interfaces in frontend/src/types/api.type.ts (e.g., AllTaskResponseType, WorkspaceByIdResponseType). Axios call functions in api.ts strictly return typed Promises (Promise).

R (Result): Achieved full compile-time autocomplete and safety. Renaming or modifying an API field immediately surface TypeScript errors during build.

Follow-up Questions & Interviewer-Expected Answers:

Q: How do you handle date serialization across the network barrier?

A: MongoDB stores dates as ISO Date objects, but JSON responses serialize them as strings. Frontend types define dates as string, and components parse them using helper functions like format(new Date(dueDate), 'MMM dd').

Q: What strategy prevents 'any' types when handling dynamic query parameters?

A: In getAllTasksQueryFn, we use structured payload interfaces (AllTaskPayloadType) with optional keys (keyword?: string, priority?: TaskPriorityEnumType). URLSearchParams appends parameters conditionally, preserving strict types.

Key Takeaway: Centralized interface contracts in api.type.ts matching backend JSON structures.

● MEDIUM (~55%) Q9: How do you optimize task listing queries for large workspaces?

■ Performance & Pagination | PREP Framework

P (Point): We combine compound MongoDB indexes with server-side pagination, selective projection, and debounced client-side filtering.

R (Reason): Fetching all tasks without limit causes query slowdowns and memory spikes as workspaces scale to thousands of items.

E (Evidence):

In getAllTasksService:

- 1) We construct dynamic filter queries over workspaceId, projectId, status, priority, assignedTo.
- 2) Execution uses .skip((pageNumber - 1) * pageSize).limit(pageSize).
- 3) Promise.all executes TaskModel.find() and TaskModel.countDocuments() concurrently.
- 4) .populate('assignedTo', '_id name profilePicture -password') excludes sensitive fields.
- 5) Compound index { workspaceId: 1, createdAt: -1 } accelerates sorting.

P (Point): This query design maintains sub-50ms API response times regardless of total collection size.

Follow-up Questions & Interviewer-Expected Answers:

Q: Why execute find() and countDocuments() in Promise.all?

A: Promise.all runs both database queries in parallel on the MongoDB driver thread pool rather than sequentially, cutting total query latency nearly in half.

Q: How does the frontend search bar avoid sending an API request on every keystroke?

A: The search bar uses a custom useDebounce hook (300ms delay) on the search input state. Query parameters update only after typing pauses, preventing search endpoint flooding.

Key Takeaway: Concurrent query execution (Promise.all), compound indexing, and debounced filters.

● MEDIUM (~50%) Q10: What measures prevent common web vulnerabilities like XSS, CSRF, and IDOR in TeamForge?

■ Security & Vulnerability Mitigation | STAR Framework

S (Situation): SaaS applications handling multi-tenant data are prime targets for cross-tenant data leaks (IDOR), token theft (XSS), and unauthorized mutations (CSRF).

T (Task): I implemented systemic defenses across authentication, input sanitization, and data access layers.

A (Action): 1) XSS: Auth JWTs stored in httpOnly cookies; React auto-escapes rendered JSX.

2) CSRF: Auth cookies set SameSite=Strict and Secure in production.

3) IDOR: Every service query explicitly includes workspaceId along with resource ID (e.g., TaskModel.findOne({ _id: taskId, workspace: workspaceId })).

4) Injection: Zod validates all inputs; Mongoose parameterizes queries.

R (Result): Eliminated IDOR vectors across all endpoints. Users cannot view or modify resources outside their authorized workspace even if they guess valid ObjectIDs.

Follow-up Questions & Interviewer-Expected Answers:

Q: How does specifying workspaceId on every resource query prevent IDOR?

A: If an endpoint queried TaskModel.findById(taskId), an attacker in Workspace A could guess a taskId from Workspace B and access it. Including workspace: workspaceId in the query ensures MongoDB returns null if the task doesn't belong to the active workspace.

Q: How does SameSite=Strict protect against CSRF attacks?

A: SameSite=Strict instructs the browser never to attach the authentication cookie to requests originating from third-party sites, making forged cross-site POST/PUT requests unauthenticated.

Key Takeaway: Tenant-scoped database queries for IDOR prevention + httpOnly SameSite=Strict cookies.

● MEDIUM (~45%) Q11: How is TeamForge containerized and deployed across environments?

■ DevOps & Infrastructure | PREP Framework

P (Point): TeamForge uses Docker Compose for reproducible local development and a decoupled cloud deployment model for production.

R (Reason): Containerizing local development ensures parity between developer machines and production, while decoupled deployment allows frontend and backend to scale independently.

E (Evidence):

Local: docker-compose.yml provisions Node.js API, MongoDB replica set (required for transactions), and Redis.
Production Frontend: Deployed on Vercel with automatic edge CDN caching.
Production Backend: Containerized Node.js API hosted on Railway/AWS EC2.
Production DB: Managed MongoDB Atlas multi-node replica set with automated backups.

P (Point): This architecture keeps dev environment setup to one command (docker-compose up) while utilizing cloud-native platforms for production.

Follow-up Questions & Interviewer-Expected Answers:

Q: Why is a MongoDB replica set required in the local docker-compose.yml environment?

A: MongoDB requires write-ahead logging (oplog) across a replica set to execute multi-document transactions. Single standalone MongoDB instances reject `startSession()` transaction calls.

Q: How are environment configurations managed across stages?

A: Environment variables are loaded via `get-env.ts` with fallback defaults. Local dev uses `.env` (gitignored), while production variables are injected securely via Railway and Vercel configuration panels.

Key Takeaway: Docker Compose local replica set + Vercel frontend + Railway/EC2 backend.

● **HIGH (~85%) Q12: Walk me through the most challenging bug or technical hurdle you solved in TeamForge.**

■ **Complex Debugging & Engineering Challenges | STAR Framework**

S (Situation): When testing workspace deletion in staging, we discovered that deleting a workspace occasionally left orphaned Task and Project documents in MongoDB if a network glitch occurred mid-operation.

T (Task): I needed to make workspace deletion completely atomic across four separate collections (Workspace, Project, Task, Member).

A (Action): I refactored `deleteWorkspaceByIdService` to initialize a Mongoose session transaction (`startSession()`). I ordered mutations carefully: verify owner rights -> delete projects (`ProjectModel.deleteMany({ workspace })`).session(session)) -> delete tasks -> delete members -> update user's currentWorkspace -> delete workspace document -> `session.commitTransaction()`. Wrapped everything in a try/catch block with `session.abortTransaction()`.

R (Result): Eliminated data corruption risks. Simulated failure injections proved that partial deletions successfully roll back 100% of mutations.

Follow-up Questions & Interviewer-Expected Answers:

Q: How did you handle the edge case where the user's currentWorkspace was the workspace being deleted?

A: Inside the session, we check if (`user.currentWorkspace.equals(workspaceId)`). We query `MemberModel.findOne({ userId })` to find another workspace the user belongs to and reassign `user.currentWorkspace` to that ID before committing.

Q: What trade-off did you evaluate when deciding to run cascading deletes on the server vs using database hooks?

A: Mongoose middleware hooks like `post('remove')` don't trigger reliably on bulk operations like `deleteMany()`. Executing explicit sequential `deleteMany()` calls within an explicit transaction guarantees execution control and atomicity.

Key Takeaway: Multi-collection cascade teardown with user context re-pointing within a Mongoose session.

● **HIGH (~80%) Q13: Why choose a Document database (MongoDB) over a Relational database (PostgreSQL) for TeamForge?**

■ **Technical Trade-offs | PREP Framework**

P (Point): MongoDB was chosen for document schema flexibility, seamless JSON/TypeScript integration, and fast nested reads.

R (Reason): TeamForge features evolving domain entities (dynamic task priorities, custom statuses, emoji metadata) where rigid relational migrations add development friction.

E (Evidence):

Pros gained:

- Schema flexibility for tasks and project metadata.
- BSON/JSON mapping directly aligns with TypeScript interfaces without complex ORM translation layers.
- High-performance read operations on primary collections.

Trade-offs mitigated:

- Relational integrity: Enforced in application service layer using explicit validation and Mongoose schema constraints.
- Joins: \$lookup used sparingly; strategic denormalization (workspace ID on Task) avoids deep joins.

P (Point): For an agile SaaS product, MongoDB delivered the best balance of speed and structure.

Follow-up Questions & Interviewer-Expected Answers:

Q: If TeamForge required complex financial reporting across tenants, would MongoDB still be suitable?

A: If cross-tenant analytical reporting and complex relational aggregations became core features, PostgreSQL with SQL joins and window functions would be superior. MongoDB aggregations become cumbersome for deep relational joins.

Q: How do you guarantee unique constraints in MongoDB without relational foreign keys?

A: We use Mongoose unique compound indexes (e.g., `memberSchema.index({ userId: 1, workspaceId: 1 }, { unique: true })`), which are enforced directly by MongoDB's B-tree index engine.

Key Takeaway: Document flexibility vs relational integrity trade-off; application-enforced constraints.

● **MEDIUM (~50%) Q14: How would you re-architect TeamForge to support 100,000 active workspaces?**

■ **Scalability & Future Architecture | STAR Framework**

S (Situation): At 100,000 active workspaces, a single database instance and monolithic Node.js process would suffer from CPU bottlenecks, database connection saturation, and slow query execution.

T (Task): Design a scaling roadmap to transition TeamForge from a single-node setup to a high-availability distributed architecture.

A (Action): I planned a 4-step scaling strategy:

- 1) Database Sharding: Shard MongoDB cluster by `workspaceId` as the shard key, ensuring all data for a workspace resides on the same shard.
- 2) Caching Layer: Redis cluster for session caching and caching `getWorkspaceById` responses.
- 3) Asynchronous Queues: BullMQ + Redis for background tasks (sending invite emails, generating analytics reports).
- 4) Stateless API Scale: Horizontal scaling of Express containers behind an AWS Application Load Balancer.

R (Result): The architecture isolates tenant workloads, ensuring a high-traffic workspace doesn't impact other tenants ('noisy neighbor' isolation).

Follow-up Questions & Interviewer-Expected Answers:

Q: Why is `workspaceId` an ideal MongoDB shard key for TeamForge?

A: Because 99% of database queries filter by `workspaceId`. Sharding by `workspaceId` ensures queries are routed to a single shard (targeted queries) rather than broadcasting across all shards (scatter-gather).

Q: How would real-time task updates be handled across multiple backend containers?

A: By implementing Socket.IO with a Redis Pub/Sub adapter. When a user updates a task on Server A, the event publishes to Redis, which broadcasts it to Server B where other workspace members are connected.

Key Takeaway: Tenant-based database sharding (workspaceId), Redis caching, and async job queues.

● **MEDIUM (~45%) Q15: What is your testing philosophy and setup for TeamForge?**

■ **Testing Strategy | PREP Framework**

P (Point): We use a multi-tiered testing strategy: Jest for backend service/unit testing and Playwright for end-to-end user flow testing.

R (Reason): Unit tests verify core business logic fast, while E2E tests guarantee critical user journeys (registration, workspace creation, task management) function in real browser environments.

E (Evidence):

Backend Testing:

- mongodb-memory-server spins up an ephemeral, in-memory MongoDB instance for fast test execution without database mocking.
- Tests cover service methods (createWorkSpaceService, changeMemberRoleService) and error paths.

Frontend Testing:

- Playwright executes automated browser sessions testing login, modal forms, and permission guards.

P (Point): This dual approach ensures both API correctness and reliable client-side user experiences.

Follow-up Questions & Interviewer-Expected Answers:

Q: Why use mongodb-memory-server instead of mocking Mongoose models with jest.mock()?

A: Mocking Mongoose models tests your mocks, not your queries. mongodb-memory-server runs real MongoDB engine binaries in memory, validating actual queries, indexes, and transactions fast without network latency.

Q: How do you test routes that require authenticated users in Jest?

A: We create a helper function that generates valid JWT tokens signed with the test JWT_SECRET. Test HTTP requests include the token header Authorization: Bearer , executing real middleware logic.

Key Takeaway: In-memory database integration testing (mongodb-memory-server) + Playwright E2E.

● **MEDIUM (~40%) Q16: How do you enforce code quality and prevent technical debt in TeamForge?**

■ **Code Hygiene & Standards | PREP Framework**

P (Point): We enforce quality programmatically using ESLint, Prettier, TypeScript strict mode, and Husky pre-commit hooks.

R (Reason): Manual code reviews should focus on architectural design and logic, not formatting style or missing type checks. Programmatic enforcement automates style consistency.

E (Evidence):

- 1) Husky + lint-staged: Intercepts git commit and runs ESLint + Prettier on staged files.
- 2) TypeScript strict mode: Rejects implicit any, unsafe index accesses, and unhandled nulls.
- 3) Zod runtime guards: Prevents malformed API requests from entering application logic.
- 4) Clean Git workflow: Feature branches merged via PRs with automated GitHub Actions CI builds.

P (Point): Automating these checks keeps the codebase maintainable and prevents lint regression over time.

Follow-up Questions & Interviewer-Expected Answers:

Q: What ESLint rules do you consider non-negotiable for TypeScript projects?

A: @typescript-eslint/no-explicit-any set to error, @typescript-eslint/explicit-module-boundary-types for public services, and no-floating-promises to catch unhandled async promises.

Q: How does lint-staged improve developer experience compared to running linter on the whole project?

A: lint-staged runs checks only on modified, staged files, completing in under 2 seconds. Running linters over the full project on every commit creates friction and slows developer velocity.

Key Takeaway: Automated pre-commit static analysis via Husky, lint-staged, and strict TypeScript.

● MEDIUM (~35%) Q17: How are runtime errors captured, logged, and surfaced to users?

■ Operational Resilience | STAR Framework

S (Situation): Unhandled server exceptions can leak stack traces to clients (security risk) or crash Node.js processes, leaving users stranded with generic browser errors.

T (Task): Build a robust error handling pipeline that sanitizes client responses while capturing detailed diagnostic logs.

A (Action): I implemented a hierarchy of custom operational error classes extending AppError (BadRequestException, UnauthorizedException, NotFoundException). Controllers throw these directly or catch runtime errors via asyncHandler. The global errorHandler logs full stack traces and returns structured JSON: { message, errorCode, statusCode }. On the client, Axios response interceptors catch error codes and display toast notifications.

R (Result): Clean client UX with 100% sanitized public error messages, while server logs retain detailed stack traces for debugging.

Follow-up Questions & Interviewer-Expected Answers:

Q: What is the difference between an operational error and a programmer error?

A: Operational errors are expected failure conditions (e.g., invalid password, user not found, 404). Programmer errors are unexpected bugs (e.g., TypeError: cannot read property of undefined). Operational errors return formatted 400-level JSON; programmer errors trigger 500 responses and process logging.

Q: How does the frontend handle a 401 Unauthorized error globally?

A: An Axios response interceptor intercepts 401 statuses globally. If a request fails with ACCESS_UNAUTHORIZED, the interceptor clears client auth state and redirects the browser to /login.

Key Takeaway: Custom AppError hierarchy, global error middleware, and Axios interceptor redirects.

● LOW (~30%) Q18: How do you handle environment configurations and secret management across environments?

■ Configuration Management | PREP Framework

P (Point): Environment variables are loaded via centralized configuration files with strict startup validation.

R (Reason): Hardcoding secrets or relying on unvalidated process.env calls scattered across files leads to silent runtime failures when an environment variable is missing.

E (Evidence):

- 1) Centralized backend/src/config/app.config.ts reads process.env variables.
- 2) A custom get-env.ts utility function asserts that required environment variables (e.g., JWT_SECRET, MONGO_URI, GOOGLE_CLIENT_ID) exist at startup. If missing, the process fails fast with an explicit error message.
- 3) .env files are strictly gitignored, with .env.example committed as a documentation template.

P (Point): Fail-fast configuration validation prevents silent failures in production deployments.

Follow-up Questions & Interviewer-Expected Answers:

Q: Why is 'failing fast' at startup better than lazy-loading environment variables when needed?

A: Lazy-loading means a missing JWT_SECRET is not discovered until a user tries to log in hours after deployment. Failing fast during container boot halts bad deployments instantly during CI/CD checks.

Q: How are frontend environment variables handled differently from backend secrets?

A: Vite bundles frontend variables prefixed with VITE_ into the static JavaScript client build. Backend variables (JWT_SECRET, MongoDB passwords) remain strictly server-side and are never exposed to the client bundle.

Key Takeaway: Centralized config modules with startup fail-fast validation.

● LOW (~25%) Q19: What are the most critical architectural improvements you would make next to TeamForge?

■ Product Strategy & Feature Prioritization | STAR Framework

S (Situation): As team usage grows, asynchronous collaboration (email-only) becomes a bottleneck, and user activity requires real-time presence.

T (Task): Identify high-value technical additions to transform TeamForge into a real-time collaborative workspace.

A (Action): I mapped out three high-priority enhancements:

- 1) WebSockets (Socket.IO): For real-time task board updates, presence indicators, and live typing feedback.
- 2) S3/Cloudinary File Attachments: Presigned URL upload pipeline for task attachments.
- 3) Audit Logging: A dedicated AuditLog collection capturing workspace configuration changes and permission updates for security reporting.

R (Result): These additions would elevate TeamForge from a basic task tracker to an enterprise-ready collaborative platform.

Follow-up Questions & Interviewer-Expected Answers:

Q: How would you securely handle S3 file uploads for task attachments?

A: Using AWS S3 Presigned URLs. The client requests an upload URL from the backend -> backend verifies user workspace permissions -> backend generates a temporary presigned PUT URL -> client uploads directly to S3. S3 credentials never touch the client.

Q: How would you structure the Audit Log collection?

A: AuditLog: { workspaceId, actorId, action (e.g. 'MEMBER_ROLE_CHANGED'), targetId, metadata, timestamp }. Indexed by workspaceId and timestamp for fast compliance querying.

Key Takeaway: Real-time WebSocket event layer, presigned S3 uploads, and audit logging.

● HIGH (~90%) Q20: Looking back, what key engineering lesson did building TeamForge teach you?

■ Engineering Leadership & Reflection | STAR Framework

S (Situation): In early development, I focused heavily on feature velocity, implementing endpoints rapidly without establishing strict API error contracts or comprehensive unit tests upfront.

T (Task): Refactoring data models and auth logic mid-project became time-consuming due to a lack of regression test safety nets.

A (Action): I paused feature development to establish strict Zod validation schemas, unified custom error classes, and built

out an integration test suite using mongodb-memory-server. I adopted a Contract-First mindset for subsequent features.

R (Result): Significantly reduced bug regression rates. I learned that upfront discipline in domain boundaries, types, and testing dramatically increases overall shipping speed in the long run.

Follow-up Questions & Interviewer-Expected Answers:

Q: How has building TeamForge changed your approach to designing software systems?

A: I now design systems with explicit domain boundaries and failure modes in mind from day one. I prioritize clear type contracts, defense-in-depth security, and testability over pure coding speed.

Q: What advice would you give an engineer starting a multi-tenant project?

A: Establish tenant boundaries (e.g., workspaceId) explicitly in data schemas and middleware at the very beginning. Retrofitting multi-tenancy and data isolation onto a single-tenant database architecture later is extremely difficult and error-prone.

Key Takeaway: Contract-First design, upfront test architecture, and explicit tenant boundary modeling.

How to Ace Your TeamForge Technical Interview

1. Master the STAR Method for Architectural & Behavioral Deep Dives

- **Situation:** State the application context in 1-2 tight sentences (e.g. multi-tenant isolation, cascade deletes).
- **Task:** State the engineering requirement or security constraint clearly.
- **Action:** Describe YOUR technical implementation details (middleware, schemas, transactions, React Query invalidations). Spend 60% of your time here.
- **Result:** Mention data integrity gains, bug elimination, or performance metrics.

2. Master the PREP Method for Technology Choices & Design Questions

- **P (Point):** State your technical choice upfront in one sentence.
- **R (Reason):** Explain WHY this tech choice fits TeamForge better than alternatives.
- **E (Evidence):** Reference explicit files, hooks, schemas, or query logic from the TeamForge codebase.
- **P (Point):** Re-state your conclusion.

3. Answer Follow-up Questions Like an SDE 1

- Avoid vague 1-line answers. Give 2-3 substantive sentences that explain the **mechanism** and the **trade-off**.
- Always tie answers back to code components (`roleGuard.ts`, `Member` collection, `isAuthenticated`, `usePermissions`).

TeamForge Core Stack Quick Reference

Layer	Technology	Implementation Detail in TeamForge
Frontend	React 18 + Vite + TS	Component architecture with strict typing
Server State	TanStack Query v5	Query invalidation on mutation + stale-while-revalidate
Client State	Zustand + React Context	Persistent auth store + usePermissions hook
Backend API	Node.js + Express + TS	MVC architecture with asyncHandler and Zod schemas
Authentication	Passport.js + JWT + OAuth	JWT in httpOnly cookie + Google OAuth account linking
Authorization	Custom RBAC Middleware	roleGuard checking RolePermissions lookup map
Database	MongoDB + Mongoose	7 collections, Mongoose Document typing, replica set
Data Consistency	MongoDB Transactions	Mongoose session transactions for cascading deletes
Testing	Jest + Playwright	In-memory MongoDB tests + browser E2E test suites